Hyperspectral and Multispectral Image Fusion for Remotely Sensed Target Detection: A New Cloud–Edge Collaborative Approach

Zebin Wu, *Senior Member, IEEE*, Chenxin Liu, Jin Sun, *Member, IEEE*, Jun Hu, Zhihui Wei, Javier Plaza, *Senior Member, IEEE*, and Antonio Plaza, *Fellow, IEEE*

***Abstract*— Hyperspectral target detection (HTD) can provide detailed information about the objects and materials within a scene and holds significant importance in remote sensing image analysis. Traditional HTD methods often suffer from low detection accuracy when applied to hyperspectral images (HSIs) with low spatial resolution, where the target only occupies a few pixels. This can be addressed by exploiting the higher spatial resolution of multispectral images (MSIs). In addition, many cloud-based HTD methods, which rely on the distributed processing capability of cloud computing to cope with large-scale datasets, may result in long transmission delays that cannot meet real-time requirements. This article suggests a cloud–edge collaborative HTD approach based on the fusion of remotely sensed HSIs and MSIs. We first introduce an HTD algorithm that employs low-rank matrix decomposition and hierarchical constraint energy minimization (hCEM) to fuse a low-resolution HSI (LR-HIS) and a high-resolution MSI (HR-MSI). Aiming at a continuous shooting scenario, we further present a cloud–edge implementation of the HTD algorithm through the collaboration of a cloud cluster and edge servers deployed close to data acquisition devices. The overall processing flow of remotely sensed data fusion in the cloud–edge environment is formulated as a flowshop scheduling-like optimization problem. We develop a co-optimization scheduling algorithm to explore the best resource allocation solutions to the formulated problem. Experimental results on both general-purpose and real-world datasets show that the newly proposed HTD algorithm leads to significant improvements in detection accuracy over traditional methods, and the cloud–edge collaborative approach further enhances computational efficiency.**

***Index Terms*— Cloud–edge collaborative computing, hyperspectral target detection (HTD), multiobjective optimization.**

Received 18 October 2024; revised 9 February 2025; accepted 11 March 2025. Date of publication 26 March 2025; date of current version 4 April 2025. This work was support in part by the National Natural Science Foundation of China under Grant U23B2006, in part by Jiangsu Provincial Innovation Support Program under Grant BZ2023046, and in part by Jiangsu Provincial Key Research and Development Program under Grant BE2022065-2. *(Corresponding author: Jin Sun.)*

Zebin Wu, Chenxin Liu, Jin Sun, Jun Hu, and Zhihui Wei are with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, Jiangsu 210094, China (e-mail: wuzb@njust.edu.cn; liuchenxin2000@njust.edu.cn; sunj@njust.edu.cn; 121106010803@njust.edu.cn; gswei@njust.edu.cn).

Javier Plaza and Antonio Plaza are with the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, 10071 Cáceres, Spain (e-mail: jplaza@unex.es; aplaza@unex.es).

Digital Object Identifier 10.1109/TGRS.2025.3554711

I. Introduction

With the recent developments in remote sensing technology, hyperspectral images (HSIs) have received widespread attention and fostered the development of advanced analysis techniques, such as spectral decomposition, segmentation, classification, and detection. Hyperspectral target detection (HTD) [1] extracts targets of interest based on the spectral differences between prior and background targets and is widely used in military applications, forest science monitoring, mineral detection, and precision agriculture [2], [3], [4]. HSIs collect photons over a large spatial range to ensure a high signal-to-noise ratio (SNR). Consequently, imaging systems typically obtain low-spatial-resolution HSIs (LR-HSIs) [5]. The idea of remotely sensed data fusion has been proposed to address the low accuracy of target detection results in LR-HSIs, resulting from the fact that a single HSI pixel may be a mixture of multiple pure spectral signatures (called endmembers in the literature). This can be addressed by exploiting the higher spatial resolution of multispectral images (MSIs). As a result, a considerable amount of studies have investigated the fusion of HSIs and MSIs for HTD (exploiting the characteristics of HSIs with high spatial resolution and the characteristics of MSIs with high spatial resolution) to improve the accuracy of HTD [6].

In the literature, there exist several different categories of HTD methods using HSI and MSI data fusion. Traditional pan-sharpening methods combine LR-HSIs with high-resolution panchromatic images. The main limitation of pan-sharpening methods is that significant distortions may occur in the case of significant differences in spatial resolution between HSIs and MSIs. On the other hand, both HSIs and MSIs can be represented as 3-D tensors. Borsoi et al. [7] proposed a tensor representation method based on Tucker decomposition which introduces variation factors between HSIs and MSIs to characterize the local spectral and spatial variations caused by different acquisition conditions. Compared to pan-sharpening methods, tensor representation methods suffer from high computational costs [8]. In addition, deep learning methods are now widely used in multisource remotely sensed data fusion [9], [10]. The main issue associated with this category of methods is that, due to differences in the number of spectral bands, spatial resolution, and spectral coverage, deep learning

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

models trained on a certain type of data only cannot be applied to other data types.

In this work, we focus on using the matrix decomposition method to implement the multisource fusion of HSI and MSI data. Matrix decomposition methods unfold 3-D images into 2-D matrices and assume that HSIs can be decomposed into the product of two matrices. This category of methods can be further classified into sparse representation methods [11], [12] and low-rank methods [13], [14], [15]. Sparse representation methods utilize the spectrum basis as a super-complete dictionary. The dictionary is learned from LR-HSIs using sparse dictionary learning algorithms. The estimates of basis function coefficients are regularized using sparse priors, typically employing sparse coding for coefficient estimation. Low-rank methods consider that spectral features can be represented by low-dimensional subspaces, learning from LR-HSIs through analysis techniques such as vertex component analysis (VCA) [16] or truncated singular value decomposition methods. Matrix decomposition methods demonstrate satisfactory HTD performance but face the high computational cost associated with solving complex optimization algorithms iteratively.

Due to the ever-increasing amount of HSI data and the inherent high computational complexity, on the other hand, many HTD methods cannot meet the requirements of computational efficiency when processing large-scale datasets. The mainstream solution is to utilize the distributed processing capability of cloud computing to cope with massive data repositories [17], [18]. However, cloud servers are mostly distant from the data acquisition devices and exhibit weaknesses such as large data transmission volume and high latency in real-time HSI applications. As a new computing paradigm, edge computing complements cloud computing by reducing latency through local data processing, which is crucial for real-time HSI applications [19], [20]. By deploying edge servers between data acquisition devices and the cloud, edge computing improves bandwidth efficiency by minimizing data transmission to the cloud. We combine clouds and edge servers to establish a cloud–edge collaborative platform that meets the requirements of high-quality and low-latency computing services in real-time application scenarios. The processing of HTD algorithms can be abstracted as a series of flow-processing subtasks. Scheduling strategies are responsible for determining the deployment of subtasks onto the computing resources provided by the cloud and edge servers [21], [22], [23].

With the aforementioned concerns in mind, this article suggests a new cloud–edge collaborative HTD approach using multisource remotely sensed data fusion. Specifically, we propose an HSI and MSI fusion algorithm that employs low-rank matrix decomposition and hCEM to perform HTD and accordingly name this algorithm LRC-HTD. Aiming at a continuous shooting scenario, we further present a cloud–edge implementation of the LRC-HTD algorithm based on the collaboration of a cloud cluster and edge servers deployed close to data acquisition devices. The overall processing flow of multisource remotely sensed data fusion in the cloud–edge environment can be formulated as a flowshop scheduling-like optimization problem. We develop a co-optimization scheduling algorithm to determine the most appropriate assignment of HTD tasks onto computing resources, reducing the execution time and energy consumption on the cloud–edge platform. In summary, the main contributions of this work are summarized as follows.

1) We propose a multisource data fusion-based HTD approach that employs low-rank matrix decomposition and the hCEM detector.
2) We establish a cloud–edge collaborative framework for the parallel implementation of the HTD approach in a continuous shooting scenario.
3) We develop a co-optimization scheduling algorithm to facilitate the efficient processing of HTD tasks on the cloud–edge platform.

Experimental results on both general-purpose and real-world datasets show that our newly proposed HTD algorithm leads to significant improvements in terms of detection accuracy over traditional methods. Moreover, our cloud–edge collaborative approach further enhances computational efficiency compared with cloud- and edge-based implementations.

This article proceeds as follows. Section II provides the preliminaries of the hCEM detector. Section III presents the multisource remotely sensed data fusion-based HTD algorithm. Section IV details the cloud–edge implementation, including the optimization model and scheduling approach. Evaluation results are given in Section V, followed by some concluding remarks in Section VI.

II. Preliminaries

In this section, we briefly introduce the fundamental flow of the hCEM method for HTD. This method is composed of multiple layers of traditional constrained energy minimization (CEM) detectors that are connected hierarchically [24]. At each layer, the background spectrum is suppressed relying on the detection result of the current layer, reducing its amplitude while preserving its direction within the spectral space. The hCEM detector is constructed through a rectified correlation matrix that contains a large amount of spectral information. As a result, hCEM can concentrate on those pixels that are difficult to detect. The CEM output of the kth layer can be represented by

$$y^k = \frac{\mathbf{R}_k^{-1} d}{d^T \mathbf{R}_k^{-1} d} \mathbf{X}^k \tag{1}$$

where $\mathbf{X}^k$ and $\mathbf{R}_k^{-1}$ represent the spectral matrix and the correlation matrix of the kth layer, respectively. Each spectral vector x_i^{k+1} in the next layer can be obtained by multiplying the input x_i^{k+1} of the current layer by a weight value q that is dependent on the output of the current layer, that is,

$$x_i^{k+1} = q(y_i^k) x_i^k. \tag{2}$$

Here, $q(t)$ is an increasing nonlinear function whose value ranges between 0 and 1, defined as

$$q(t) = \begin{cases} 1 - e^{-\lambda t}, & \text{if } t \geq 0 \\ 0, & \text{if } t < 0. \end{cases} \tag{3}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

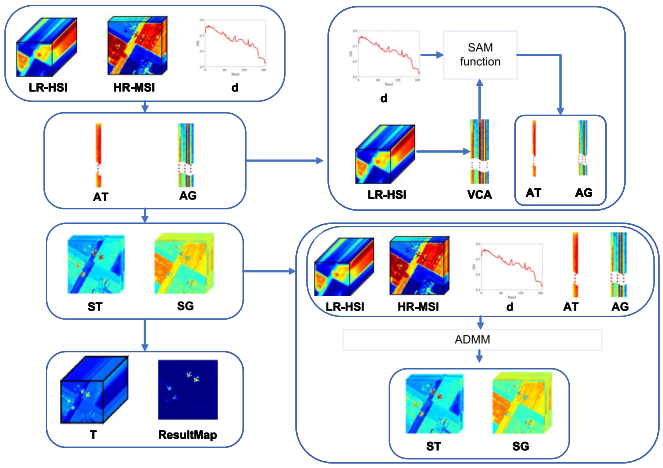


Fig. 1. General flowchart of the LRC-HTD method.

The weight value can be considered as a soft threshold operation to preserve the spectra with higher output scores and suppress those with lower output scores. In this manner, unrelated background spectra will be gradually suppressed through each detection layer, whereas the target spectrum will remain unchanged.

The matrix $\mathbf{R}_k$ in (2) is defined as

$$\mathbf{R}_k = \frac{1}{N}\mathbf{X}_k\mathbf{X}_k^T. \tag{4}$$

The output of each layer y^k can be obtained according to (2) and (4). The error δ_k is then calculated as the difference between the average output energy of the current layer and that of the previous layer, that is,

$$\delta_k = \frac{1}{N}\left\|y^k\right\|_2^2 - \frac{1}{N}\left\|y^{k-1}\right\|_2^2. \tag{5}$$

The aforementioned layer-by-layer calculation procedure is terminated once the error δ_k is less than a sufficiently small threshold value. The hCEM detector not only enhances detection efficiency but also preserves the advantages of CEM in terms of computational efficiency.

III. LRC-HTD Algorithm

This section details our newly proposed LRC-HTD approach that employs low-rank matrix decomposition and the hCEM detector for multisource remotely sensed data fusion. In our LRC-HTD, $\mathbf{X}$ represents a high-resolution HSI (HR-HSI), $\mathbf{Y}$ represents an LR-HSI, and $\mathbf{Z}$ represents an HR-MSI. The degradation procedure of $\mathbf{Y}$ and $\mathbf{Z}$ can be formulated as

$$\mathbf{Y} = \mathbf{XBH} + \mathbf{E_Y} \tag{6}$$

$$\mathbf{Z} = \mathbf{RX} + \mathbf{E_Z} \tag{7}$$

where $\mathbf{B} \in \mathbb{R}^{N_W N_H \times N_W N_H}$ and $\mathbf{H} \in \mathbb{R}^{N_W N_H \times N_W N_h}$ denote a spatial fuzzy operator and a spatial downsampling operator, respectively, and $\mathbf{R} \in \mathbb{R}^{N_b N_B}$ stands for the spectral response of the multispectral sensor. In addition, $\mathbf{E_Y}$ and $\mathbf{E_Z}$ represent random modeling errors.

A general processing flowchart of our proposed LRC-HTD is given in Fig. 1. To achieve fusion (and subsequent target detection) of HSIs and MSIs in a unified model, $\mathbf{X}$ can be first separated into target $\mathbf{T}$ and background $\mathbf{G}$. Relying on the degradation process in (6) and (7), we can establish the following prior model consisting of two fidelity terms:

$$\min_{\mathbf{T},\mathbf{G}}\left\{\frac{1}{2}\|\mathbf{Y} - (\mathbf{T}+\mathbf{G})\mathbf{BH}\|_F^2 + \frac{1}{2}\|\mathbf{Z} - \mathbf{R}(\mathbf{T}+\mathbf{G})\|_F^2\right\}. \tag{8}$$

Assuming that the target and background of HSIs are located in different spectral subspaces, the endmembers (pure spectral signatures) of $\mathbf{X}$ should be composed of two parts: one part spanning the target subspace and the other part spanning the background subspace. Then, we have

$$\mathbf{X} = \mathbf{A_T}\mathbf{S_T} + \mathbf{A_G}\mathbf{S_G} + \mathbf{E} \tag{9}$$

where $\mathbf{A_T} \in \mathbb{R}^{\mathbf{B} \times \mathbf{k_1}}$ and $\mathbf{S_T} \in \mathbb{R}^{\mathbf{k_1} \times \mathbf{W} \times \mathbf{H}}$ are the endmember matrix and the abundance matrix of target $\mathbf{T}$, respectively. $\mathbf{A_G} \in \mathbb{R}^{\mathbf{B} \times \mathbf{k_2}}$ and $\mathbf{S_G} \in \mathbb{R}^{\mathbf{k_2} \times \mathbf{W} \times \mathbf{H}}$ are the endmember matrix and abundance matrix of $\mathbf{G}$, respectively. To solve the prior model in (8), we need to estimate, in turn, the aforementioned matrices, for which the calculation details are given below.

A. Estimating the Endmember Matrix A

Ideally, $\mathbf{A_T}$ should be a prespecified prior spectrum. However, the information carried by a single spectral feature is limited. Therefore, we first enhance the ability of $\mathbf{A_T}$ to represent the target subspace. Since the spectral information of $\mathbf{Z}$ is mainly contained in $\mathbf{X}$, it can be estimated that the spectral information $\mathbf{A} \in \mathbb{R}^{\mathbf{B} \times \mathbf{k}}$ is

$$\mathbf{A} = \rho_{\mathrm{VCA}}(\mathbf{X}) \tag{10}$$

where $\rho_{\mathrm{VCA}}(\cdot)$ represents the VCA function [15], and $\mathbf{k} = \mathbf{k_1} + \mathbf{k_2} + \mathbf{1}$. By initializing $\mathbf{A_T}$ as the given prior spectrum and $\mathbf{A_G}$ as the empty set, for $\forall i \in [1,\ \mathbf{k}]$, we preliminarily separate the spectral representations of the target and background as follows:

$$\begin{cases} \mathbf{A_T} = [\mathbf{A_T}, \mathbf{A}(:i)], & \text{if } \boldsymbol{\rho}_{\mathrm{SAM}}(\mathbf{d}, \mathbf{A}(:i)) \le \gamma \\ \mathbf{A_G} = [\mathbf{A_G}, \mathbf{A}(:i)], & \text{otherwise} \end{cases} \tag{11}$$

where $\rho_{\mathrm{SAM}}(\mathbf{x_1}, \mathbf{x_2})$ calculates the spectral angle (used to measure the adjacency of two pixels) and γ is a threshold value for the spectral angle. The separation of the target endmember matrix and background endmember matrix is beneficial for the enhancement of HTD performance.

B. Estimating the Abundance Matrix S

Given the estimated endmember matrices $\mathbf{A_T}$ and $\mathbf{A_G}$, obtained in the previous step, the following prior regularization model can be established to estimate the corresponding abundance matrices $\mathbf{S_T}$ and $\mathbf{S_G}$

$$\min_{\mathbf{S_T},\mathbf{S_G}}\left\{\frac{1}{2}\|\mathbf{Y} - (\mathbf{A_T}\mathbf{S_T} + \mathbf{A_G}\mathbf{S_G})\mathbf{BH}\|_F^2, + \frac{1}{2}\|\mathbf{Z} - \mathbf{R}(\mathbf{A_T}\mathbf{S_T} + \mathbf{A_G}\mathbf{S_G})\|_F^2 + \lambda\mathbf{R}([\mathbf{S_T}, \mathbf{S_G}])\right\} \tag{12}$$

where $\mathbf{R}([\mathbf{A_T}, \mathbf{A_G}])$ is a regularization term. We consider the low-rank regularization method that is commonly used for

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

image-processing tasks. Specifically, $\mathbf{R}(\cdot)$ is defined as the kernel norm based on local blocks and is given by

$$\mathbf{R}(\mathbf{Z}) = \sum_{\mathbf{m}=1}^{\mathbf{M}} \left\| \mathbf{R}_{(\mathbf{m})} \right\|_* \tag{13}$$

where $\left\| \mathbf{R}_{(\mathbf{m})} \right\|_*$ represents the nuclear norm of $\mathbf{R}_{(\mathbf{m})}$, that is, the sum of nonzero singular values of the mth local block matrix. The preceding operations, in fact, simplify the process of multisource data fusion and target detection, which only requires optimizing two variables: $\mathbf{S_T}$ and $\mathbf{S_G}$ after estimating the endmember matrix. Most importantly, applying low-rank terms to the abundance matrix of the HR-HSI (rather than to the background abundance matrix) can ensure better utilization of low-rank information, resulting in better super-resolution results.

C. Solving the Target Matrix T

We multiply $\mathbf{A_T}$ derived from (11) by $\mathbf{S_T}$ in (12) to determine the target matrix

$$\mathbf{T} = \mathbf{A_T} * \mathbf{S_T}. \tag{14}$$

The last step is to use the hCEM method to detect targets in $\mathbf{T}$ and obtain the final result.

Algorithm 1 summarizes the computational procedure of our proposed LRC-HTD. The input includes the LR-HSI $\mathbf{Y}$, the HR-MSI $\mathbf{Z}$, and the prior spectrum $\mathbf{d}$, as well as the weight value λ and spectral angle threshold γ. The output is the final result map produced by the hCEM detector. In brief, LRC-HTD improves background suppression in the target matrix by refining the target endmember matrix and calculating the corresponding target abundance matrix.

Algorithm 1 LRC-HTD

Input: LR-HSI $\mathbf{Y}$, HR-MSI $\mathbf{Z}$, prior spectrum $\mathbf{d}$, γ, and λ

Output: result map **ResMap**

1 Calculate $\mathbf{A} = \rho_{\mathrm{VCA}}(\mathbf{X})$;
2 Calculate the similarity between $\mathbf{d}$ and each spectrum in $\mathbf{A}$;
3 Use all spectra that are similar to $\mathbf{d}$ and the prior spectrum $\mathbf{d}$ to construct $\mathbf{A_T}$;
4 Determine $\mathbf{S}$ by iteratively solving Eq. (12);
5 Decompose S to obtain $\mathbf{S_T}$ and $\mathbf{S_G}$;
6 Calculate $\mathbf{T} = \mathbf{A_T} * \mathbf{S_T}$;
7 Use hCEM to detect $\mathbf{T}$ and generate **ResMap**;
8 **return ResMap**;

IV. Cloud–Edge Collaborative Approach

We now present the cloud–edge collaborative implementation that we have developed to accelerate the proposed LRC-HTD approach on a cloud–edge platform consisting of a cloud cluster and a set of edge servers. We formulate the processing flow of LRC-HTD (in a continuous shooting scenario) as a combinatorial optimization problem that is analogous to the well-known flowshop scheduling problem [25]. We suggest an evolutionary computation-based co-optimization scheduling algorithm to solve the formulated problem by determining the optimized assignment of cloud and edge resources for performing HTD tasks.

A. Cloud–Edge Collaborative Model

Considering an uncrewed aerial vehicle (UAV)-assisted scenario in which data acquisition devices deployed on UAVs continuously capture HSIs, the architecture of the cloud–edge collaborative computing system is depicted in Fig. 2. The cloud–edge system consists of a set of UAVs as data acquisition terminals, several edge servers that are close to the terminals, and a cluster of virtual machines (VMs) provided by the cloud server. Let J_i ($i \in \{0, 1, \ldots, I\}$) represent the ith HTD task, $O_{i,j}$ ($j \in \{0, 1, \ldots, J\}$) represent the jth operation of J_i, and M_k ($k \in \{0, 1, \ldots, K\}$) denote the kth device. We propose formulating the parallel processing of HTD tasks in this continuous shooting scenario as a flowshop scheduling-like optimization problem. The analog between the cloud–edge collaborative problem and the flowshop scheduling problem can be explained as follows. Similar to the jobs that need to be assigned to machines in a flowshop, the HTD tasks are to be scheduled onto the computing resources in the cloud–edge system. In our LRC-HTD approach, each task involves a series of operations, including a CEM-based coarse detection operation, a spectral response operation, an endmember matrix refinement operation, an abundance matrix estimation operation, and a target matrix fine detection operation. Any of these subtasks is assigned to a certain computing node, which can be an edge server or a cloud VM, for execution, and the subtasks can be executed in parallel on the cloud server. Just as flowshop scheduling aims to minimize the overall makespan of job processing, the cloud–edge collaborative problem attempts to achieve the goal of optimizing certain performance metrics (total completion time and energy consumption in this work) when processing continuous HTD tasks.

To handle these HTD tasks efficiently, it is necessary to select appropriate devices for each task depending on parameters such as the amount of data to be processed, the computation workload, and the transmission overheads. For a certain HSI, the assignment of its computation task and the transmission between the cloud and edge server are coupled. The transmission delay, as well as the energy consumption caused by the cloud–edge interaction, would be dependent on which edge server the computation task is assigned to. To this end, we introduce a cloud–edge collaborative optimization model for efficient processing of the multisource remotely sensed data fusion flow to jointly minimize the task completion time and energy consumption.

1) Edge Computing Model: The data acquisition terminal collects HSI and MSI data at certain intervals and offloads the collected data to edge servers for subsequent processing. At the data offloading stage, for a specific HTD task J_i, the transmission delay for offloading the HSI data to the edge

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

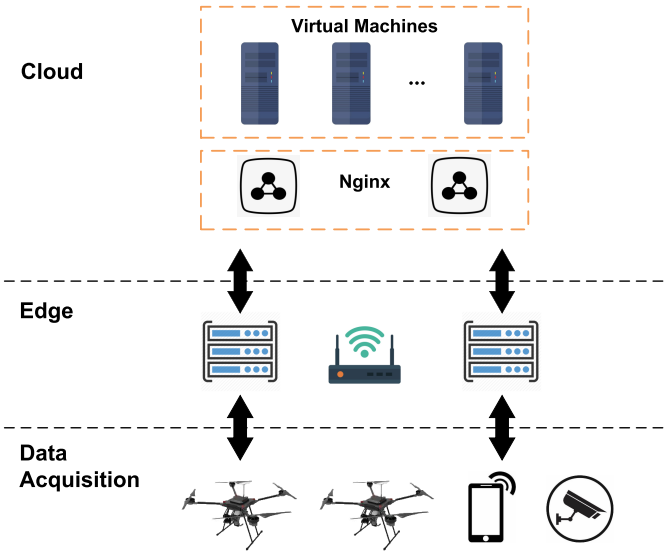


Fig. 2. Cloud–edge collaborative computing architecture.

server can be represented by

$$T^{i}_{\text{offload}} = \frac{H_i}{B_1} \tag{15}$$

where H_i represents the amount of collected data and B_1 represents the transmission rate from the data acquisition device to the edge server (which is assumed to be constant during the data offloading).

At the edge computing stage, if there are any available computing resources on the edge server when the data offloading is completed, the HTD task can start execution. The execution time of $O_{i,j}$ (i.e., the jth subtask of J_i) on the edge server is determined by

$$T^{i,j}_{\text{EdgeExe}} = \frac{D_{i,j}}{f_{\text{edge}}} \tag{16}$$

where $D_{i,j}$ stands for the computation workload of $O_{i,j}$ (in CPU cycles) and f_{edge} represents the computing frequency of the edge server. The starting time for each task is as follows:

$$T^{i,j}_{\text{start}} = \begin{cases} C_{\text{terminal}}, & \text{if } i = 1 \\ T^{i-1,j}_{\text{finish}}, & \text{otherwise} \end{cases} \tag{17}$$

where C_{terminal} is the time elapsed by the data acquisition device in collecting and offloading the HSI and MSI data. For the first subtask executing on the edge server, it can start execution immediately after the data are ready for processing. Any other subtask needs to wait until the execution of its predecessor task (denoted by $T^{i-1,j}_{\text{finish}}$) has been completed. The completion time of the subtask is determined by its start time and execution time and can be denoted by

$$T^{i,j}_{\text{finish}} = T^{i,j}_{\text{start}} + T^{i,j}_{\text{EdgeExe}}. \tag{18}$$

The execution time for all subtasks on the edge server is the maximum value among all completion times, that is,

$$T^{i}_{\text{edge}} = \max_{j}\left\{T^{i,j}_{\text{finish}}\right\}. \tag{19}$$

The time elapsed on the edge server by the ith HTD task includes the transmission time and edge execution time

$$C^{i}_{\text{edge}} = T^{i}_{\text{offload}} + T^{i}_{\text{edge}}. \tag{20}$$

2) Cloud Computing Model: In the cloud–edge collaborative system, the cloud server provides a certain number of VMs, on which the HTD tasks can be executed in parallel. The edge server further uploads the results of completed subtasks to the cloud server for parallel execution of remaining subtasks. The released computing resources of the edge server can continue to process new subtasks offloaded from the data acquisition side. The computing process of cloud servers includes the data upload stage and cloud computing stage. At the stage of edge-to-cloud transmission, due to the varying execution progress of subtasks for different suspicious targets on edge servers, the amount of the data that needs to be uploaded to the cloud server also varies and can be determined as

$$T_{\text{trans}} = \sum_{j=1}^{J} \frac{H_{m,j}}{B_2} \tag{21}$$

where index j indicates the jth suspicious target ($0 < j < J$) and $H_{m,j}$ represents the transmission amount of the jth suspicious target offloaded to the cloud in the mth task. The value of m depends on how many subtasks of the jth suspicious target have been completed on the edge server. B_2 represents the transmission rate between the edge server to the cloud server.

At the cloud execution stage, a subtask can immediately start execution once its associated data has arrived and there are available computing resources on the cloud server. The execution time of subtask $O_{i,j}$ on the cloud VM is given by

$$T^{i,j}_{\text{CloudExe}} = \frac{D_{i,j}}{f_{\text{cloud}}} \tag{22}$$

where $D_{i,j}$ represents the amount of data associated with subtask $O_{i,j}$ and f_{cloud} represents the computing frequency of the edge server. In addition, the start time of $O_{i,j}$ can be denoted by

$$T^{i,j,k}_{\text{start}} = \begin{cases} T^{i}_{\text{edge}}, & \text{if } i = 1 \\ \max\left\{T^{i-1,j}_{\text{finish}},\ T^{i,j,k}_{\text{finish}}\right\}, & \text{otherwise} \end{cases} \tag{23}$$

where $p < i$ and $q < j$. The first subtask arriving at the cloud VM starts its execution at the moment when the edge server has completed uploading the data of task J_i to the cloud server, that is, T^{i}_{edge}. Each remaining subtask cannot start execution until its predecessor task on the same VM completes execution. Accordingly, the finish time of $O_{i,j}$ on the kth cloud VM can be expressed as

$$T^{i,j,k}_{\text{finish}} = T^{i,j,k}_{\text{start}} + \frac{D_{i,j}}{f_{\text{cloud}}}. \tag{24}$$

The completion time of each calculation task is the sum of the task start time and the calculation time. We use a queue T_{Pre} to record all completed tasks in a sequential order. Once a subtask completes its execution, it will be added to T_{Pre} and its

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

completion time will be recorded. The final completion time for all subtasks of J_i on the cloud server is

$$T^i_{\text{cloud}} = \max_{j,k}\left\{T^{i,j,k}_{\text{finish}}\right\}. \tag{25}$$

As such, the time elapsed on the cloud server by the ith task would be

$$C^i_{\text{cloud}} = \sum_{j=1}^{J} \frac{H_{m,j}}{B_2} + T^i_{\text{cloud}}. \tag{26}$$

3) Energy Consumption Model: The total energy consumption of the cloud–edge implementation takes into account the energy costs consumed by the data acquisition device, edge, and cloud servers and that by data transmission. Assuming that the average power of the data acquisition device is P_{collect}, the energy consumption of the data acquisition device E_{collect} can be expressed as

$$E_{\text{collect}} = C_{\text{terminal}} \times P_{\text{collect}}. \tag{27}$$

The energy cost the data transmission, denoted by P_{transmit}, is related to the transmission rate B_1 and data size H_1 and can be denoted by

$$E_{\text{trans}} = P_{\text{trans}} \times \frac{H_1}{B_1} \tag{28}$$

where P_{trans} represents the transmission power.

The energy consumption of the edge server in processing a computational task depends on its computing power and the amount of computation workload involved in task execution. If the average power of the edge server is P_{edge}, the energy consumption E_{EdgeExe} associated with the jth suspicious target of the ith task is

$$E_{\text{edge}} = P_{\text{edge}} \times \frac{D_{i,j}}{f_{\text{edge}}}. \tag{29}$$

In the cloud–edge collaborative paradigm, the cloud server acts as a resource-constrained private cloud server. Similarly, the energy consumption of the cloud server when processing HTD tasks depends on its computing power and the computation workload, which is given by

$$E_{\text{cloud}} = P_{\text{cloud}} \times \frac{D_{i,j}}{f_{\text{cloud}}} \tag{30}$$

where P_{cloud} is the average operating power for executing tasks on cloud VMs.

The aforementioned energy components are summed to obtain the total energy consumption E_{total}

$$E_{\text{total}} = E_{\text{collect}} + E_{\text{trans}} + \sum_{i,j} E_{\text{edge}} + \sum_{k} E_{\text{cloud}}. \tag{31}$$

4) Co-Optimization Model: Under HSI and MSI data streams, we consider the processing flow of HTD tasks on the cloud–edge collaborative platform as a co-optimization problem that jointly minimizes the total competition time and energy consumption. The objective is to determine the most appropriate assignment of HTD tasks onto the edge servers and cloud VMs such that the two performance metrics are minimized in a Pareto-optimal sense [26]. The result of the co-optimization problem is a Pareto frontier consisting of all nondominated solutions, rather than a single solution. Specifically, the co-optimization is summarized as

$$\min \quad \sum_{i=1}^{m} T_i \tag{32}$$

$$\min \quad E_{\text{total}} \tag{33}$$

$$\text{subject to} \quad T^{i-1,j}_{\text{finish}} \le T^{i,j}_{\text{start}} \tag{34}$$

$$C_{\text{edge}} < \min\left\{T^{i,j,k}_{\text{start}}\right\} \tag{35}$$

$$\sum_{k=1}^{K} T \le \mu \tag{36}$$

$$0 < i \le I,\ 0 < j \le J,\ 0 < k \le K \tag{37}$$

where $T_1 = C_{\text{terminal}}$, $T_2 = C_{\text{edge}}$, and $T_3 = C_{\text{cloud}}$. Equations (34) and (35) denote the constraints upon the order of task execution on the edge servers and cloud VMs, respectively. Equation (36) imposes a constraint upon the cloud's computing resources, in which μ stands for the maximum number of available VMs on the cloud server.

B. Co-Optimization Scheduling Approach

According to the preceding problem formulation, the optimal processing of the LRC-HTD approach on the cloud–edge collaborative platform is similar to solving the flowshop scheduling problem, which is NP-hard [27]. To this end, we introduce a co-optimization scheduling algorithm built upon a multiobjective genetic algorithm (GA) [28] to explore an optimized solution to the formulated optimization problem, that is, the optimal assignment of HTD tasks onto cloud–edge computing resources. The proposed scheduling algorithm mainly involves three procedures: solution encoding, solution evaluation, and nondominated sorting-based scheduling.

1) Encoding Scheme: To facilitate the implementation of the scheduling algorithm, a primary step is to link every individual in the GA population to a feasible scheduling solution. Since a chromosome individual is typically encoded as a string of genes, we design an integer encoding scheme that depends on the state of edge servers and cloud VMs to enable the mapping of chromosome individuals onto scheduling solutions. Specifically, the encoding scheme uses the following task sequence to represent a valid solution to the co-optimization problem:

$$\mathcal{Z} = \left[\zeta_{1,1}, \zeta_{1,2}, \ldots, \zeta_{i,j}, \ldots, \zeta_{n,m}\right]. \tag{38}$$

Assuming that there are n tasks, where each consists of m subtasks to detect suspicious targets, the total length of the sequence would be $n \times m$. A specific subtask $\zeta_{i,j}$ denotes the jth suspicious target of the ith task. The positions of all subtasks in the encoded sequence represent their order of execution.

This encoding scheme specifies the correspondence between HTD tasks and cloud–edge computing resources, which reduces the difficulty of selection, crossover, and mutation operations. In addition, the encoded sequence ensures that a single task is assigned to only one VM, which satisfies the corresponding scheduling constraints. When the computational

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

tasks of the cloud–edge collaborative computing architecture are changed, only the length of the task sequence needs to be changed. In brief, our proposed encoding method satisfies the diverse characteristics of devices in the cloud–edge collaborative computing platform and is simple to implement while exhibiting good scalability.

2) Solution Evaluation: The encoding scheme establishes a mapping relation between any chromosome in the GA framework and a task sequence representing a valid scheduling solution. We use the following steps to evaluate the quality of a specific candidate solution denoted by $[T_1, T_2, \ldots, T_m]$. The algorithmic flow of solution evaluation is summarized in Algorithm 2, which is to allocate all the subtasks in the sequence to cloud or edge servers such that the completion time and energy consumption can be obtained. Using an illustrative example of a cloud–edge collaborative system with two data acquisition devices and three edge servers, the procedure for evaluation of solution quality can be described as follows.

Algorithm 2 Solution Evaluation

Input: a candidate solution $[T_1, T_2, \ldots, T_m]$
Output: completion time $\sum_{i=1}^{m} T_i$ and energy consumption E_{total}

Initialize T_{pre}, T_{exe}, and T_{finish};
Collect data on acquisition device and add T_0 to T_{pre};
foreach *subtask* T_i *executed on the edge server* **do**
  if T_i *is the first subtask* **then**
    Set $T_{\text{finish}}[i-1] \leftarrow C_{\text{terminal}}$;
  Calculate T_{para} according to Eq. (16);
  Set $T_{\text{exe}}[i-1] \leftarrow T_{\text{para}}$;
  Set $T_{\text{finish}}[i-1] \leftarrow T_{\text{finish}}[i-1] + T_{\text{para}}$;
  Add the completed subtask $T_{1,1}$ to T_{pre};
  Calculate E_{edge} according to Eq. (29);
Calculate C_{edge} according to Eq. (20);
foreach *subtask* T_j *executed on the cloud VM* **do**
  Set $T_{\text{finish}}[j-1] \leftarrow C_{\text{edge}}$;
  if *the predecessor task of* T_j *is in* T_{pre} **then**
    Calculate T_{trans} according to Eq. (21);
  Calculate E_{trans} according to Eq. (28);
  Calculate T_{para} according to Eq. (22);
  Set $T_{\text{exe}}[j-1] \leftarrow T_{\text{para}}$;
  Set $T_{\text{finish}}[j-1] \leftarrow T_{\text{finish}}[j-1] + T_{\text{para}}$;
  Add the completed subtask to T_{pre};
  Calculate E_{cloud} according to Eq. (30);
Calculate C_{cloud} according to Eq. (26);
Calculate the completion time $\sum_{i=1}^{m} T_i$ and energy consumption E_{total};
return $\sum_{i=1}^{m} T_i$ and E_{total};

Step 1: Initialize $T_{\text{pre}} = \phi$, $T_{\text{exe}} = [0, 0, \ldots, 0]$, and $T_{\text{finish}} = [0, 0, \ldots, 0]$, where T_{pre} represents the queue of completed subtasks, T_{exe} represents execution time and T_{finish} represents task completion time. Also, we use T_{para} to denote the parallel processing time of a subtask. The value of T_{para} depends on the computing resource that is assigned for executing the subtask and will be determined after this subtask has been scheduled.

Step 2: The first step is to collect data on the acquisition device. Add T_0 to the completed subtask queue T_{pre}. The execution time is the data collection interval, and the task completion time is C_{terminal}.

Step 3: According to subtasks $(T_1, T_6, \ldots, T_{5k+1})$ executed at the edge server in the task sequence, the number of subtasks executed at the edge server for each HTD task is obtained. T_{para} should be the subtask's execution time on the edge server and can be calculated using (16). The execution time of the edge server is $T_{\text{exe}}[0] = T_{\text{para}}$ and the completion time is $T_{\text{finish}}[0] = T_{\text{finish}}[0] + T_{\text{para}}$. The first task at the edge needs to be executed after data acquisition is completed, so the initial $T_{\text{finish}}[0]$ is C_{terminal}. Add the completed subtasks $T_{1,1}$ to T_{pre}. Calculate the energy consumption E_{Edge} using (29).

Step 4: Traverse the task sequence and determine whether the task to be executed on the edge server has completed execution, and go to Step 2 otherwise. The task completion time at the edge server is C_{edge}.

Step 5: According to subtasks $(T_2, \ldots, T_5, T_7, \ldots, T_m)$ executed by the cloud VM in the task sequence, the number of subtasks executed by each task on the VM is obtained. The task start time on the cloud VM is C_{edge}, that is, $T_{\text{finish}}[k] = C_{\text{edge}}$.

Step 6: If the predecessor task of the subtask is included in T_{pre} on the edge server, the cloud–edge transmission overhead needs to be taken into account. The transmission energy consumption E_{trans} can be obtained by (28).

Step 7: Calculate T_{para} using (22), that is, the subtask's execution time on the cloud VM. The execution time of the cloud VM is $T_{\text{exe}}[k] = T_{\text{para}}$, and the completion time is $T_{\text{finish}}[k] = T_{\text{finish}}[k] + T_{\text{para}}$. Then, add the completed subtasks $T_{2,1}$ to the $PreT$. Calculate the computation energy consumption E_{cloud} at the edge server according to (30).

Step 8: Traverse the task sequence, and determine whether all tasks are completed or not, otherwise go to Step 5, task completion time is C_{cloud}, energy consumption is E_{cloud}.

Step 9: Use C_{cloud}, C_{edge}, C_{terminal}, E_{collect}, C_{transmit}, C_{Edge}, and C_{Cloud} values obtained in previous steps to calculate the total completion time and energy cost of the cloud–edge collaborative system.

3) Nondominated Sorting-Based Scheduling: Following the multiobjective GA framework in [28], we develop a nondominated sorting-based scheduling algorithm that incorporates an elitism strategy to retain high-quality individuals in the next-generation population and improve the scheduling algorithm's convergence to an optimized solution. The general flow of the scheduling algorithm is summarized in Fig. 3.

We start with encoding all chromosomes in the population into task sequences and calculate the objective values of all sequences. Then, we use nondominated sorting to evaluate the quality of all chromosome individuals. Depending on the dominance relations among individuals, we assign all individuals in the population with corresponding priority levels. The nondominated solutions on the Pareto frontier would have higher priority levels. The nondominated solution set is initialized as an empty set and is assigned level 1.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

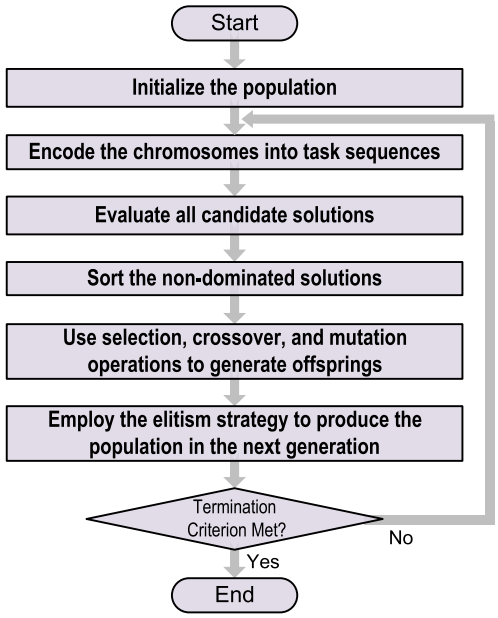


Fig. 3. Flowchart of the co-optimization GA.

We iterate over each individual in the population and identify all its dominated solutions. If the number of dominated solutions n is 0, we set level 1 for this individual and add it to the nondominated solution set with level 1. We check all individuals in the nondominated solution set with the current level. For each individual A, we check every single individual B in its dominated solution set and subtract the number of individual B's dominated solution by 1. If there are no dominated solutions for individual B, we set B's level to $l+1$ and add it to the nondominated solution set with the next level.

Once we have sorted all the nondominated solutions, we use two GA operators, crossover and mutation, to generate the offspring of current individuals in the population. We select any two chromosomes from the population as parent chromosomes. A point from parent chromosome 1 is randomly selected as the crossover point, and all subtask sequences located before the crossover point in parent chromosome 1 remain unchanged, and task sequences after the crossover point are emptied. The subtasks in the sequence of the parent chromosome 2 that are not covered by the sequence before the crossover point of the parent chromosome 1 are filled into the vacant positions of the parent chromosome 1 in accordance with the original order. The mutation operation randomly introduces new chromosomes to prevent the search from falling into the local optimum. This operation further enhances the diversity of the population and enhances the global search capability by randomly selecting a feasible sequence to replace a subsequence of the original chromosome.

The co-optimization scheduling algorithm maintains a population of chromosomes that evolves over many generations. When the offspring population has been generated in the preceding procedure, we further employ an elitism strategy to ensure that high-quality chromosome individuals can be retained in the next generation. We combine the parent and offspring populations, identify all infeasible solutions, and exclude them from the combined population. Then, we sort the remaining feasible solutions in descending order of priority level and select the top individuals to form the subsequent generation. Such a selection mechanism in the elitism strategy can address the excessive reproduction issue in population evolution and prevent the degradation in population quality, beneficial for accelerating the convergence rate and improving the performance of the scheduling algorithm.



Fig. 4. HSIs provided in Houston 2018 dataset.

V. Experimental Results

This section presents the evaluation results. We use both general-purpose and real-world datasets to verify the detection performance of the newly proposed LRC-HTD approach. We also establish a cloud–edge collaborative system to examine whether the cloud–edge implementation can substantially enhance the computational efficiency of the LRC-HTD by using the co-optimization scheduling algorithm.

A. HSI Datasets

To validate the effectiveness of the proposed LRC-HTD approach based on HSI and MSI data fusion, we take into account four widely used datasets: San Diego I, San Diego II, Gulfport, and Los Angeles [29]. After removing low-SNR bands, we retain 186 bands for San Diego I and San Diego II, as well as 191 and 205 spectral bands for the Gulfport and Los Angeles datasets. Each dataset used in the experiment has a size of 100 $\times$ 100 pixels, with airplanes serving as targets. Also, we use the real-world Houston 2018 dataset [30], in which yellow and red vehicles are identified as targets, to assess the LRC-HTD approach in practical application. To obtain the LR-HSI, we use a Gaussian filter to blur the HR-MSI on a band-by-band basis and downsample the blurred image by selecting one pixel out of every Gaussian filter [31]. The spatial downsampling ratio between the LR-HSI and HR-MSI, which is referred to as the scaling factor (SF), is set to 5 and 10. In addition, the LRC-HTD approach is based on the hCEM detector and requires prior spectral information. The prior spectrum is obtained by randomly selecting three spectra from the target pixels and averaging them. The area under the curve (AUC) value resulting from receiver-operating characteristics (ROC) analysis is used to evaluate the detection results. A higher AUC value indicates better target detection accuracy. Before performance evaluation, the grayscale values of all datasets have been normalized into the range [0, 1].

To evaluate the performance of the cloud–edge collaborative implementation of LRC-HTD and the co-optimization scheduling algorithm, we used the real-world Houston 2018 dataset to simulate a continuous UAV shooting scenario. Fig. 4 shows the HSI provided in this dataset. The size

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

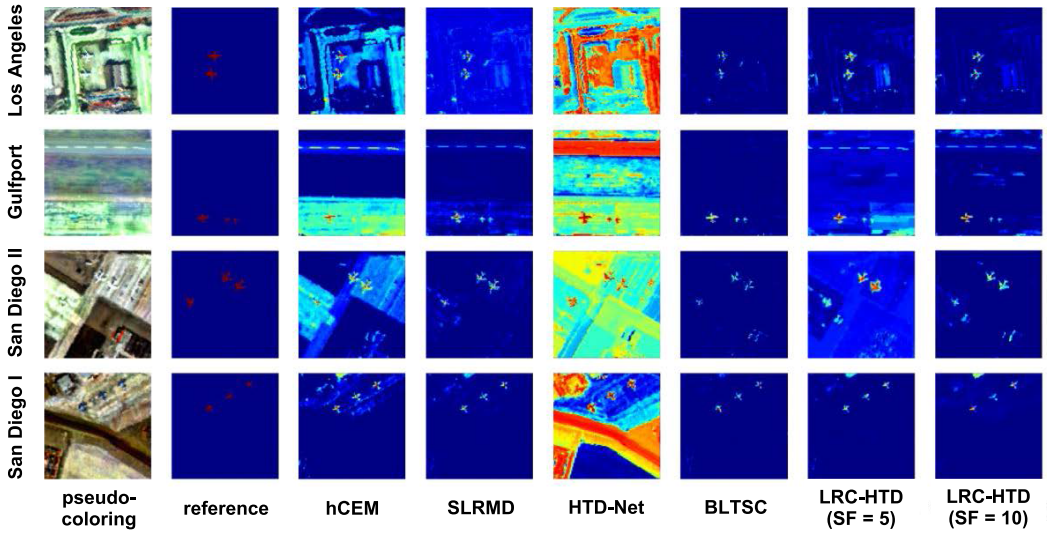


Fig. 5. Fusion results obtained by LRC-HTD and baseline approaches.

of the captured images was 601 × 596 for the HSI and 12 020 × 11 920 for the MSI, with a shooting interval of 60 s. The first shot was aimed at yellow vehicles with three suspicious target regions. The second shot and third shots were aimed at red vehicles, with four and five suspicious target regions, respectively. Then, we periodically generate the HSI and MSI data at these shooting moments to simulate the process of continuous shooting.

B. Experimental Results on General-Purpose Datasets

We first examine the performance of our proposed LRC-HTD approach on general-purpose datasets. The baseline HTD approaches used for comparison are model-based and learning-based methods, including sparse and low-rank factorization method (SLRMD) [32], hCEM [24], convolutional neural network (HTD-Net) [33], and background learning-based autoencoder network (BLTSC) [29]. Traditional target detection methods operate on HR-HSIs. In contrast, the LRC-HTD approach detects targets in downsampled LR-HSIs and HR-HSIs by incorporating low-rank matrix decomposition and hCEM detector to leverage multisource remote sensing fusion. Fig. 5 presents the resulting map obtained by baseline approaches and LRC-HTD using two SF values of 5 and 10, respectively. We can observe that LRC-HTD achieves the most accurate target detection and demonstrates superior background pixel suppression, even when the original image has low spatial resolution. The BLTSC method, which incorporates background learning, delivers the highest AUC value due to its effective suppression of background pixels. Although other comparison methods are capable of detecting targets, they exhibit less effective background suppression.

Table I presents the detection results in terms of the AUC value obtained by different methods on all datasets. The AUC results indicate that our proposed LRC-HTD significantly performs the baseline approaches on the Los Angeles dataset and also achieves satisfactory performance on the other three datasets. LRC-HTD using SF = 10 achieves the highest AUC value on Los Angeles, San Diego I, and San Diego II datasets. When the SF value is 5, the detection accuracy slightly decreases in comparison to when the SF value is 10 but still can outperform most baseline algorithms. Only on the Gulfport dataset, the AUC value from the traditional hCEM detector is slightly higher than that from LRC-HTD. However, as can be observed in Fig. 5, our proposed LRC-HTD leads to much better background suppression than hCEM does.

C. Experimental Results on Real-World Datasets

The performance of the LRC-HTD approach is also evaluated on the real-world Houston 2018 dataset. In this set of experiments, we use an SF value of 20 for the downsampling ratio. The HSIs of dimension 601 × 596 and MSIs of dimension 12 020 × 11 920 are selected as reference images. Fig. 6(a) shows the detection results obtained using yellow vehicles as the target. During coarse detection of the reference image, three suspicious target regions are obtained and each target region is fused for target detection. Similarly, Fig. 6(b) presents the detection results obtained using red vehicles as the target, indicating that four suspicious target regions are obtained. We can also observe from Fig. 6 that our proposed LRC-HTD achieves clearer detection results and better background suppression effect than the baseline approaches do. When multiple targets appear in certain suspicious target regions, LRC-HTD is capable of detecting all targets. The improvement of detection performance can be attributed to the refinement of the endmember matrix during the estimation process, which enhances the background suppression capability of LRC-HTD in the target matrix.

D. Simulation Results of Continuous Shooting

We now present the evaluation results obtained by using the cloud–edge implementation of LRC-HTD in the continuous shooting scenario. Fig. 7 presents the simulation results of the data acquisition device consecutively capturing six HSIs, with each color representing a specific HTD task. The three rows (from top to bottom) represent the data acquisition device,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
AUC RESULTS OBTAINED BY LRC-HTD AND BASELINE APPROACHES

Dataset	hCEM	SLRMD	HTD-Net	BLTSC	LRC-HTD (SF = 5)	LRC-HTD (SF = 10)
San Diego I	0.9797	0.9503	0.8981	0.9192	0.9939	0.9963
San Diego II	0.9855	0.9917	0.8833	0.9891	0.9899	0.9940
Gulfport	0.9977	0.9594	0.9860	0.9704	0.9887	0.9922
Los Angeles	0.9222	0.8404	0.7174	0.8771	0.9436	0.9793

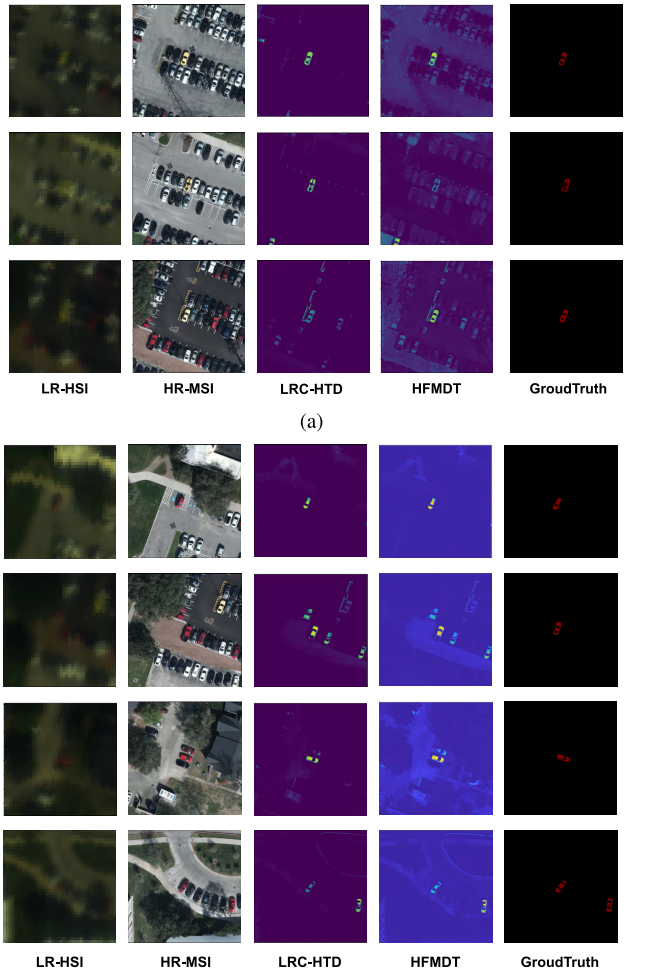


Fig. 6. Detection results on the Houston 2018 dataset. (a) Yellow vehicles as the target. (b) Red vehicles as the target.

edge servers, and cloud VMs, respectively. Specifically, the acquisition device collects HSIs and MSIs every 60 s. The edge server processes the data and obtains suspicious target regions once data collection has been completed. We use the proposed co-optimization scheduling algorithm to allocate computing tasks between edge servers and cloud VMs. After the tasks on the edge servers are completed, the remaining tasks are uploaded to the cloud VMs for execution and the edge servers receive the data collected by the next acquisition device. In certain situations, for example, when the data acquisition for "Task3" is completed, the edge server is still processing the previous task. Then, "Task3" has to wait until the server is available before it can start execution.

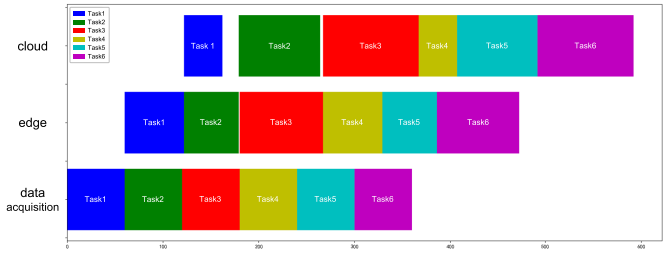


Fig. 7. Simulation results in the continuous shooting scenario.

TABLE II
PARAMETER SETTINGS OF SCHEDULING ALGORITHMS

Algorithm	Parameter	Value
CS	abandoning probability	0.25
	step scaling factor	2/10/1270
PSO	population size	100
	self-cognitive coefficient	1.5
	social-cognitive coefficient	1.5
	shrinkage factor	2
	inertial weight	1
GA	cross probability	0.95
	mutation probability	0.2

E. Co-Optimization Scheduling Performance

Furthermore, the co-optimization scheduling algorithm for the cloud–edge collaborative implementation is compared with other multiobjective optimization algorithms. To this end, we incorporate two representative evolutionary computation algorithms, cuckoo search (CS) [34] and particle swarm optimization (PSO) [35], into the co-optimization scheduling framework to establish their multiobjective implementations. For every single scheduling algorithm in the experiments, we set the maximum number of iterations to 100, the population size to 100, and the number of cloud VMs to 3. The relevant parameters of the three scheduling algorithms used for evaluation are listed in Table II. With the number of suspicious targets fixed at eight, we evaluate the performance of our co-optimization scheduling algorithm when compared to multiobjective CS and PSO.

As shown in Fig. 8, red dots represent the Pareto frontier (i.e., the nondominated solution set) obtained by the co-optimization scheduling algorithm, whereas blue dots represent the results obtained by PSO and the green dots represent the results obtained by CS. The comparison results show that our scheduling algorithm produces better co-optimization solutions compared with PSO and CS. Under the same conditions, our scheduling algorithm can find solutions

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

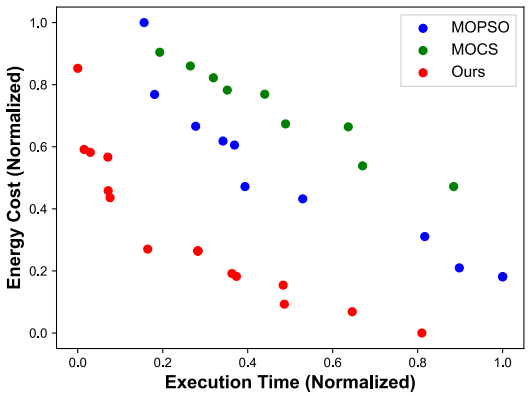


Fig. 8. (Normalized) nondominated solutions obtained by different co-optimization scheduling algorithms.

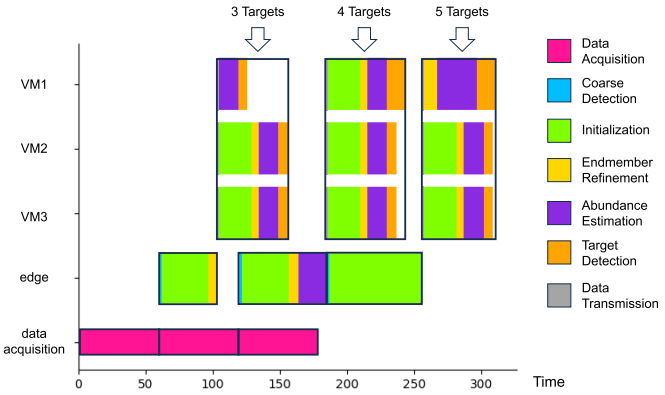


Fig. 9. HTD task execution with different numbers of suspicious targets.

TABLE III
CO-OPTIMIZATION METRICS BY USING DIFFERENT SCHEDULING ALGORITHMS

Algorithm	Ours	MOPSO	MOCS
Cardinality	14	10	9
Hypervolume	0.7724	0.6864	0.6064

TABLE IV
EXECUTION TIMES (IN SECONDS) UNDER VARYING NUMBERS OF SUSPICIOUS TARGETS

#Suspicious Targets	three	four	Five
Data Acquisition	60	60	60
Edge	49	70	77
Cloud	52	59	54
Total	161	189	191
Average (Per Target)	53.6	44.7	38.2

with lower energy costs and shorter task completion times. Furthermore, we use two commonly used co-optimization performance metrics, cardinality, and hypervolume, to evaluate the quality of the nondominated solutions quantitatively. Specifically, cardinality refers to the number of nondominated solutions produced by the co-optimization algorithm to form the Pareto frontier, and hypervolume is defined as the volume of the space covered by the nondominated solutions. The evaluation results by using different scheduling algorithms are provided in Table III. The greater the value of these two metrics, the higher the quality of the co-optimization solutions. Our scheduling algorithm outperforms the two competing algorithms in terms of both cardinality and hypervolume. Thus, we conclude that the co-optimization scheduling algorithm in this work exhibits applicability and efficiency in processing HTD tasks on the cloud–edge collaborative platform.

We also perform experiments to examine the scheduling performance considering different numbers of suspicious targets in the scene. Fig. 9 illustrates the optimal deployment of HTD tasks assigned by the scheduling algorithm for continuous detection of three, four, and five suspicious target regions. Each color block represents one of the subtasks including data acquisition, data transmission, coarse detection, initialization, endmember refinement, abundance estimation, and target detection. The data acquisition device collects HSIs every 60 s and uploads HSI data to the edge server for processing. It can be observed that the subtasks assigned to the edge server and the cloud VM are different with the varying number of suspicious targets. Note that data transmission time is incurred only when the HSI data are transmitted to the cloud via the edge server. Due to the relatively small amount of data that needs to be uploaded, the time elapsed for data transmission is negligible and not visible in Fig. 9. Table IV further provides the record of execution time at each stage obtained by the co-optimization scheduling algorithm. The execution time statistics include data acquisition time, the time elapsed at the edge server and that at the cloud VM, total execution time, and average time per target. As the number of suspicious targets increases, the total execution time does not exhibit a linear increase. The reason is that the edge servers undertake the majority of the computational burden for the suspicious targets to be fused for detection. The scheduling solution fully utilizes the computing resources of edge servers, demonstrating the advantage of the cloud–edge collaborative implementation in terms of resource utilization.

We finally compare the deployment solution obtained from the co-optimization scheduling algorithm with the results from different deployment strategies. The data acquisition device collects HSI data every 60 s, with each collected HSI considered an HTD task. Due to dataset limitations, we simulate the continuous shooting scenarios by repeatedly collecting HSIs containing 3, 4, and 5 suspicious targets. Table V records the execution times by using different deployment strategies. The execution times by deploying all tasks on the cloud VM ("Cloud-Only") and all on edge servers ("Edge-Only") as well as the execution time by using the deployment solution of our scheduling algorithm are compared. In the case of processing a single HTD task, the deployment strategy of executing all tasks on the edge server leads to the shortest execution time. In contrast, the execution time with all tasks executing on the cloud is longer due to the large amount of collected data and in turn the significant transmission overhead. As the number of tasks increases, the execution time by the "Cloud-Only" strategy is superior to that by using the "Edge-Only" strategy, since the transmission overhead accounts for a relatively small part of the total execution time. It is noticeable that the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EXECUTION TIMES (IN SECONDS) BY USING DIFFERENT DEPLOYMENT STRATEGIES

Number of Tasks	one	three	six	twelve
Cloud-Only	142	530	1060	2120
Edge-Only	82	921	1842	3684
Ours	161	321	517	909

execution time achieved by our co-optimization scheduling algorithm decreases significantly when processing more HTD tasks. When the number of consecutive tasks exceeds three, our scheduling algorithm leads to a significant reduction in execution time compared with "Cloud-Only" and "Edge-Only" deployment strategies. The comparison results justify that the computational efficiency of processing continuous HTD tasks can be substantially enhanced by fully exploiting the computing resources of the cloud–edge collaborative system.

VI. CONCLUSION

This article introduces a fusion-based HTD algorithm that employs low-rank matrix decomposition and the hCEM detector for the fusion of HSI and MSI data, as well as a distributed and parallel implementation of our proposed LRC-HTD algorithm on a cloud–edge collaborative computing platform. We model the collaborative processing of continuous HTD tasks as a flowshop scheduling-like optimization problem and develop a co-optimization scheduling algorithm to jointly minimize the execution time and energy consumption by determining the assignment of HTD tasks onto cloud–edge computing resources. Experimental results demonstrate not only the detection accuracy of the LRC-HTD algorithm but also the computational efficiency of the cloud–edge collaborative implementation.

REFERENCES

[1] B. Chen, L. Liu, Z. Zou, and Z. Shi, "Target detection in hyperspectral remote sensing image: Current status and challenges," *Remote Sens.*, vol. 15, no. 13, p. 3223, Jun. 2023.

[2] D. R. A. D. Almeida et al., "Monitoring restored tropical forest diversity and structure through UAV-borne hyperspectral and LiDAR fusion," *Remote Sens. Environ.*, vol. 264, Oct. 2021, Art. no. 112582.

[3] L. Zhang and L. Zhang, "Artificial intelligence for remote sensing data analysis: A review of challenges and opportunities," *IEEE Geosci. Remote Sens. Mag.*, vol. 10, no. 2, pp. 270–294, Jun. 2022.

[4] S. Wang et al., "Using soil library hyperspectral reflectance and machine learning to predict soil organic carbon: Assessing potential of airborne and spaceborne optical soil sensing," *Remote Sens. Environ.*, vol. 271, Mar. 2022, Art. no. 112914.

[5] X. Liu, Q. Liu, and Y. Wang, "Remote sensing image fusion based on two-stream fusion network," *Inf. Fusion*, vol. 55, pp. 1–15, Mar. 2020.

[6] G. Vivone, "Multispectral and hyperspectral image fusion in remote sensing: A survey," *Inf. Fusion*, vol. 89, pp. 405–417, Jan. 2023.

[7] R. A. Borsoi, C. Prévost, K. Usevich, D. Brie, J. C. M. Bermudez, and C. Richard, "Coupled tensor decomposition for hyperspectral and multispectral image fusion with inter-image variability," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 702–717, Apr. 2021.

[8] R. Dian, S. Li, B. Sun, and A. Guo, "Recent advances and new guidelines on hyperspectral and multispectral image fusion," *Inf. Fusion*, vol. 69, pp. 40–51, May 2021.

[9] J.-F. Hu, T.-Z. Huang, L.-J. Deng, T.-X. Jiang, G. Vivone, and J. Chanussot, "Hyperspectral image super-resolution via deep spatiospectral attention convolutional neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 7251–7265, Dec. 2022.

[10] D. Wang, L. Zhuang, L. Gao, X. Sun, M. Huang, and A. J. Plaza, "PDB-SNet: Pixel-shuffle downsampling blind-spot reconstruction network for hyperspectral anomaly detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5511914.

[11] R. Dian, S. Li, L. Fang, and Q. Wei, "Multispectral and hyperspectral image fusion with spatial–spectral sparse representation," *Inf. Fusion*, vol. 49, pp. 262–270, Sep. 2019.

[12] X. Han, J. Yu, J.-H. Xue, and W. Sun, "Hyperspectral and multispectral image fusion using optimized twin dictionaries," *IEEE Trans. Image Process.*, vol. 29, pp. 4709–4720, 2020.

[13] W. Sun et al., "MLR-DBPFN: A multi-scale low rank deep back projection fusion network for anti-noise hyperspectral and multispectral image fusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522914.

[14] Q. Wei, J. Bioucas-Dias, N. Dobigeon, and J.-Y. Tourneret, "Hyperspectral and multispectral image fusion based on a sparse representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 7, pp. 3658–3668, Jul. 2015.

[15] J. Liu, Z. Wu, L. Xiao, J. Sun, and H. Yan, "A truncated matrix decomposition for hyperspectral image super-resolution," *IEEE Trans. Image Process.*, vol. 29, pp. 8028–8042, 2020.

[16] J. M. P. Nascimento and J. M. Bioucas-Dias, "Vertex component analysis: A fast algorithm to unmix hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 4, pp. 898–910, Apr. 2005.

[17] J. Sun et al., "An efficient and scalable framework for processing remotely sensed big data in cloud computing environments," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4294–4308, Jul. 2019.

[18] J. M. Haut, S. Moreno-Alvarez, R. Pastor-Vargas, A. Perez-Garcia, and M. E. Paoletti, "Cloud-based analysis of large-scale hyperspectral imagery for oil spill detection," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 2461–2474, 2024.

[19] L. Yin, J. Sun, and Z. Wu, "An evolutionary computation framework for task off-and-downloading scheduling in mobile edge computing," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 22399–22412, Dec. 2024.

[20] R. Wang, Y. Hao, Y. Miao, L. Hu, and M. Chen, "RT3C: Real-time crowd counting in multi-scene video streams via cloud-edge-device collaboration," *IEEE Trans. Services Comput.*, vol. 17, no. 4, pp. 1739–1752, Aug. 2024.

[21] B. Kar, W. Yahya, Y. Lin, and A. Ali, "Offloading using traditional optimization and machine learning in federated cloud-edge-fog systems: A survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1199–1226, 2nd Quart., 2023.

[22] Z. Wu et al., "Scheduling-guided automatic processing of massive hyperspectral image classification on cloud computing architectures," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3588–3601, Jul. 2021.

[23] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 238–256, Jan. 2024.

[24] Z. Zou and Z. Shi, "Hierarchical suppression method for hyperspectral target detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 1, pp. 330–342, Jan. 2016.

[25] H. Emmons and G. Vairaktarakis, *Flow Shop Scheduling: Theoretical Results, Algorithms, and Applications*. Berlin, Germany: Springer, 2012.

[26] I. Giagkiozis and P. J. Fleming, "Methods for multi-objective optimization: An analysis," *Inf. Sci.*, vol. 293, pp. 338–350, Feb. 2015.

[27] M. R. Garey, D. S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Math. Oper. Res.*, vol. 1, no. 2, pp. 117–129, 1976.

[28] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[29] W. Xie, X. Zhang, Y. Li, K. Wang, and Q. Du, "Background learning based on target suppression constraint for hyperspectral target detection," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 5887–5897, 2020.

[30] S. Pande and B. Banerjee, "Adaptive hybrid attention network for hyperspectral image classification," *Pattern Recognit. Lett.*, vol. 144, pp. 6–12, Apr. 2021.

[31] R. Li, L. Zhang, Z. Wang, and X. Li, "FCSwinU: Fourier convolutions and Swin transformer UNet for hyperspectral and multispectral image fusion," *Sensors*, vol. 24, no. 21, p. 7023, Oct. 2024.

[32] A. W. Bitar, L. Cheong, and J. Ovarlez, "Sparse and low-rank matrix decomposition for automatic target detection in hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5239–5251, Aug. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.

[33] G. Zhang, S. Zhao, W. Li, Q. Du, Q. Ran, and R. Tao, "HTD-Net: A deep convolutional neural network for target detection in hyperspectral imagery," *Remote Sens.*, vol. 12, no. 9, p. 1489, May 2020.
[34] X. S. Yang and S. Deb, "Engineering optimisation by cuckoo search," *Int. J. Math. Model. Numer. Optim.*, vol. 1, no. 4, p. 330, 2010.
[35] R. E. J. Kennedy, "Particle swarm optimization," in *Proc. Int. Conf. Neural Netw.*, Perth, WA, Australia, 1995, pp. 1942–1948.



Zebin Wu (Senior Member, IEEE) received the B.Sc. and Ph.D. degrees in computer science and technology from Nanjing University of Science and Technology, Nanjing, China, in 2003 and 2007, respectively.

He was a Visiting Scholar with the Department of Mathematics, University of California at Los Angeles, Los Angeles, CA, USA, from August 2016 to September 2016 and from July 2017 to August 2017; the GIPSA-Laboratory, Grenoble INP, University of Grenoble Alpes, Grenoble, France, from August 2018 to September 2018; and the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, Escuela Politécnica, University of Extremadura, Cáceres, Spain, from June 2014 to June 2015. He is currently a Professor with the School of Computer Science and Engineering, Nanjing University of Science and Technology. His research interests include hyperspectral image (HIS) processing, parallel computing, and big data processing.



Chenxin Liu received the B.S. degree in computer science and technology from Nanjing University of Science and Technology, Nanjing, China, in 2023, where he is currently pursuing the M.Sc. degree with the School of Computer Science and Engineering.

His research interests include hyperspectral image (HIS) processing and high-performance computing.



Jin Sun (Member, IEEE) received the B.Sc. degree in computer science and technology from Nanjing University of Science and Technology, Nanjing, China, in 2004, and the Ph.D. degree in electrical and computer engineering from the University of Arizona, Tucson, AZ, USA, in 2011.

From January 2012 to September 2014, he was with Orora Design Technologies Inc., Bellevue, WA, USA, as a member of Technical Staff. He is currently a Professor with the School of Computer Science and Engineering, Nanjing University of Science and Technology. His research interests include cloud computing, edge computing, edge intelligence, and services computing.



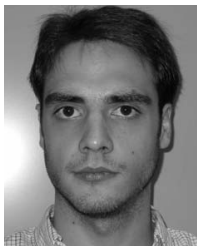
Jun Hu received the B.Sc. degree in network engineering and the M.Sc. degree in computer science and technology from Nanjing University of Science and Technology, Nanjing, China, in 2021 and 2024, respectively.

He is currently with China Construction Bank Corporation, Beijing, China as a Software Development Engineer. His research interests include hyperspectral image (HIS) processing and high-performance computing.



Zhihui Wei received the B.Sc. and M.Sc. degrees in applied mathematics and the Ph.D. degree in communication and information systems from Southeast University, Nanjing, China, in 1983, 1986, and 2003, respectively.

He is currently a Professor and a Doctoral Supervisor with Nanjing University of Science and Technology, Nanjing. His research interests include partial differential equations, image processing, multiscale analysis, sparse representation, and compressive sensing.



Javier Plaza (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in computer engineering from the University of Extremadura, Cáceres, Spain, in 2004 and 2008, respectively.

He is currently a member of the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura. He has authored more than 200 publications, including more than 80 JCR journal articles, ten book chapters, and 100 peer-reviewed conference proceedings articles. His main research interests comprise hyperspectral data processing and parallel computing of remote sensing data.

Dr. Plaza was a recipient of the Outstanding Ph.D. Dissertation Award at the University of Extremadura in 2008, the Best Column Award of *IEEE Signal Processing Magazine* in 2015, and the Most Highly Cited Paper from 2005 to 2010 of the *Journal of Parallel and Distributed Computing*. He received the best paper awards at the IEEE International Conference on Space Technology and the IEEE Symposium on Signal Processing and Information Technology. He has guest-edited four special issues on hyperspectral remote sensing for different journals. He is also an Associate Editor for IEEE GEOSCIENCE AND REMOTE SENSING LETTERS and IEEE REMOTE SENSING CODE LIBRARY.



Antonio Plaza (Fellow, IEEE) received the M.Sc. and Ph.D. degrees in computer engineering from the Department of Technology of Computers and Communications, University of Extremadura, Badajoz, Spain, in 1999 and 2002, respectively.

He is currently the Head of the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain. He has authored more than 430 journal articles, 25 book chapters, and 330 peer-reviewed conference proceedings articles. His research interests include hyperspectral data processing and parallel computing of remote sensing data.

Dr. Plaza was a member of the Editorial Board of IEEE Geoscience and Remote Sensing Newsletter from 2011 to 2012 and *IEEE Geoscience and Remote Sensing Magazine* in 2013. He is also a member of the Steering Committee of the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS). He was a recipient of the recognition of Best Reviewers of IEEE GEOSCIENCE AND REMOTE SENSING LETTERS in 2009 and IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING in 2010, the Best Column Award of *IEEE Signal Processing Magazine* in 2015, the 2013 Best Paper Award of the JSTARS, the Most Highly Cited Paper in *Journal of Parallel and Distributed Computing* from 2005 to 2010, and the Best Paper Awards at the IEEE International Conference on Space Technology and the IEEE Symposium on Signal Processing and Information Technology. He served as the Director of Education Activities for the IEEE Geoscience and Remote Sensing Society (GRSS) from 2011 to 2012 and the President of the Spanish Chapter of IEEE GRSS from 2012 to 2016. He has been identified as a Highly Cited Researcher by Clarivate Analytics from 2018 to 2023. He has guest-edited 17 special issues on hyperspectral remote sensing for different journals. He has reviewed more than 500 manuscripts for more than 50 different journals. He served as an Associate Editor for IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING. He is also an Associate Editor of IEEE ACCESS.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 16:40:32 UTC from IEEE Xplore. Restrictions apply.